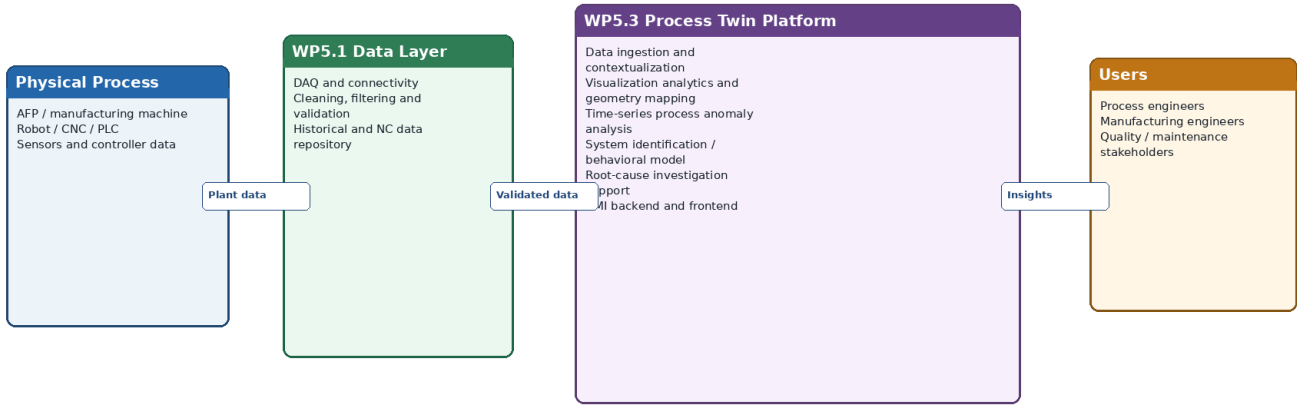


WP5.3 PROCESS TWIN & HMI

Software Architecture and Technical Design Specification

Work Package: WP5.3 Process Optimization
Architecture style: arc42-aligned
Version 1.0 | July 2026

WP5.3 PROCESS TWIN & HMI - SYSTEM CONTEXT



Scope boundary: WP5.3 consumes validated data and provides process understanding, anomaly analysis

Architecture scope overview

Document Control

Document title	WP5.3 Process Twin & HMI - Software Architecture and Technical Design Specification
Version	1.0
Status	Architecture baseline for review
Work package	WP5.3 Process Optimization
Primary audience	Process engineers, software architects, HMI developers, data engineers, analytics engineers and project stakeholders
Scope baseline	Process data analysis, time-series anomaly analysis, system identification, root-cause investigation support and HMI visualization

Revision History

Version	Date	Author/Owner	Description
1.0	July 2026	WP5.3 Team	Initial formal architecture and design baseline

Approval

Role	Name	Approval/Date
WP5.3 Work Package Owner		
Software Architecture Reviewer		
Industrial Process Stakeholder		

Table of Contents

1. Introduction and Goals
 2. Architecture Constraints
 3. System Scope and Context
 4. Solution Strategy
 5. Building Block View
 6. Runtime View
 7. Deployment View
 8. Cross-Cutting Concepts
 9. Architectural Decisions
 10. Quality Requirements
 11. Risks and Technical Debt
 12. Detailed Backend Architecture
 13. Detailed Frontend Architecture
 14. Analytics and Process Twin Services
 15. Data and Interface Architecture
 16. Testing and Validation Strategy
 17. Release and Evolution Plan
 18. Glossary
- Appendix A. Reference Architecture Diagrams
- Appendix B. Example API and Message Contracts

1. Introduction and Goals

1.1 Purpose of the Document

This document defines the software architecture and detailed technical design of the WP5.3 Process Twin and HMI platform. It establishes a common engineering baseline for implementation, integration, verification and long-term maintenance. The specification follows the structure and intent of the arc42 architecture template while extending it with implementation-level descriptions of the backend, frontend, analytics, data contracts and deployment model.

The platform is intended to transform validated process data into actionable process understanding. It combines engineering data analysis, spatial mapping, time-series process anomaly analysis, data-driven system identification, root-cause investigation support and interactive visualization. The term Process Twin is used to describe this integrated digital representation of process execution and behavior; it does not imply a physics-based simulation of the robot, machine or aircraft structure.

1.2 Business and Engineering Goals

The principal engineering goal is to provide a coherent view of process execution using data acquired by WP5.1. Engineers shall be able to inspect KPI evolution, reconstruct actual TCP trajectories, compare programmed and executed paths, superimpose process variables on part geometry, identify abnormal temporal behavior and compare actual process response with expected behavior derived from empirical models.

The platform shall shorten the time required to understand process variation and investigate abnormal conditions. It shall make process information accessible through a unified HMI rather than through disconnected logs, database queries and offline scripts. It shall also provide a reusable architecture in which basic visualization can be released before advanced time-series and system-identification modules are completed.

1.3 Scope

The architecture includes the WP5.3 data-consumption interfaces, data-ingestion adapters, contextualization services, visualization analytics, geometry mapping, time-series anomaly-analysis services, behavioral-model services, root-cause investigation support, orchestration, HMI backend APIs and HMI frontend application. Historical and NC-program data are obtained through RXU.Data or the WP5.1 repository, while live process data are consumed through RXU Connectivity topics or services.

The architecture excludes direct machine control, robot motion control, raw sensor-driver development, primary DAQ filtering owned by WP5.1, closed-loop parameter adjustment, runtime autonomous learning, virtual commissioning and physics-based environment simulation. Root-cause functionality is defined as engineering investigation support unless its diagnostic accuracy is separately validated.

1.4 Stakeholders

Stakeholder	Primary concern
Process and manufacturing engineers	Understanding KPI variation, process events, spatial behavior and potential contributors to abnormal conditions.
HMI users	Clear navigation, responsive dashboards, reliable 3D visualization and traceable investigation workflows.

WP5.1 data team	Stable data contracts, clear ownership boundaries and controlled data-consumption interfaces.
Software developers	Modular components, testable interfaces, versioned APIs and maintainable deployment.
Data and analytics engineers	Reproducible datasets, model lifecycle separation and traceable analysis outputs.
Project management	Incremental releases, visible value, bounded scope and measurable acceptance criteria.

2. Architecture Constraints

The platform is constrained by the availability and maturity of production data. Actual WP5.1 laying data is expected later than the initial WP5.3 development start; therefore the architecture must support sample mTorres data for early development and actual production data for later refinement. Data contracts shall be stable enough to avoid rebuilding the platform when the data source changes.

Constraint	Architectural consequence
WP5.1 owns acquisition, cleaning, filtering and validation	WP5.3 consumes validated data and performs contextualization, mapping, analysis and visualization without duplicating primary DAQ processing.
No physics-based 3D process model	The Process Twin is empirical and data-centric; geometry is used for visualization and spatial association, not for physical simulation.
No closed-loop control in the current scope	The platform provides insights and investigation context but does not command machine parameters.
Incremental HMI releases are required	Frontend, backend and visualization services must operate with mock or basic data before advanced analytics are available.
IPC/edge deployment	Services must run reliably within industrial-PC resource, security and network constraints.
RXU-based integration	Internal services shall use established RXU topics/services or compatible adapters where practical.

3. System Scope and Context

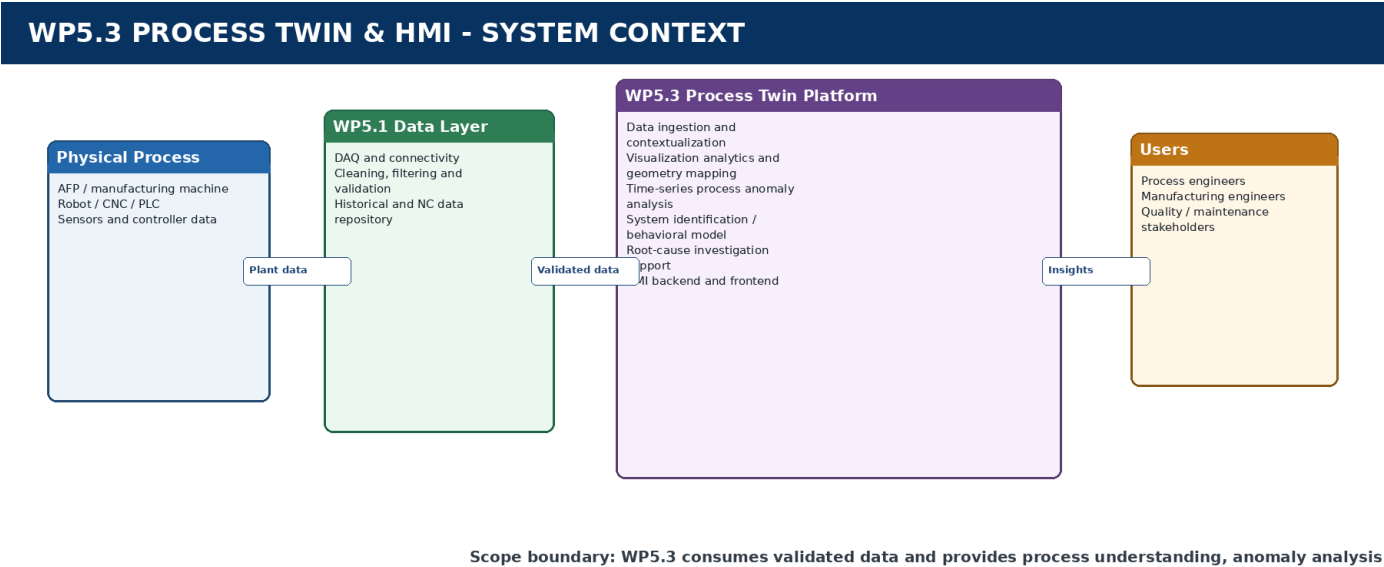


Figure 1 - WP5.3 system context

The physical process, machine controllers, robot controllers and sensors are outside the WP5.3 software boundary. Their data are acquired and validated by WP5.1 and made available through RXU.Data and RXU Connectivity. WP5.3 consumes this information, applies process-specific analysis, produces visualization-ready outputs and exposes them through an HMI hosted on an industrial PC or accessed from an authorized engineering workstation.

3.1 External Interfaces

External system	Direction	Information exchanged	Interface
RXU.Data / WP5.1 repository	Inbound	Historical KPI, TCP, events, NC program, run and part metadata	Repository API, database adapter or RXU service
RXU Connectivity	Inbound	Live KPI, machine state, TCP pose and process events	RXU topics/services
Part geometry repository	Inbound	CAD or mesh representation and coordinate metadata	File or geometry service
HMI users	Bidirectional	Queries, filters, visualization payloads and job status	HTTPS REST and secure WebSocket
Optional enterprise systems	Bidirectional/optional	Reports, data export or integration status	Controlled IT interface

4. Solution Strategy

The solution is decomposed into data services, visualization analytics, advanced analytics, orchestration, backend APIs and frontend modules. This separation prevents the HMI from becoming tightly coupled to databases or analytical algorithms. It also allows each service to evolve independently and enables an early HMI release using mock or basic datasets.

The architecture uses request-response communication for historical queries and analysis jobs, publish-subscribe communication for live process updates, and standardized JSON contracts between the backend and frontend. The orchestration layer coordinates multi-service workflows and protects the frontend from internal service sequencing and failure-handling complexity.

The implementation shall prefer explicit service contracts, versioned schemas, traceable job identifiers and deterministic historical analysis. Advanced analytics modules are independently deployable and are integrated through adapters so that a statistical or machine-learning implementation can be changed without redesigning the HMI.

5. Building Block View

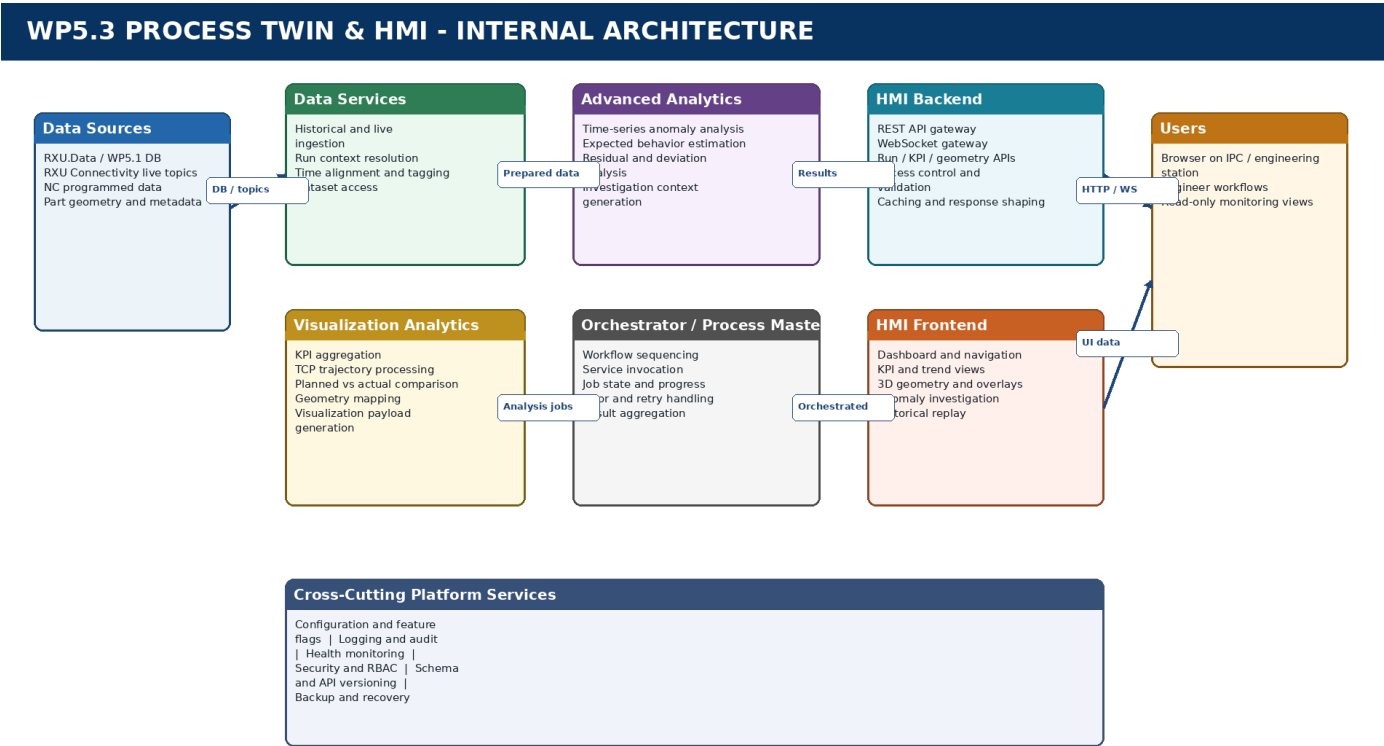


Figure 2 - Internal building-block architecture

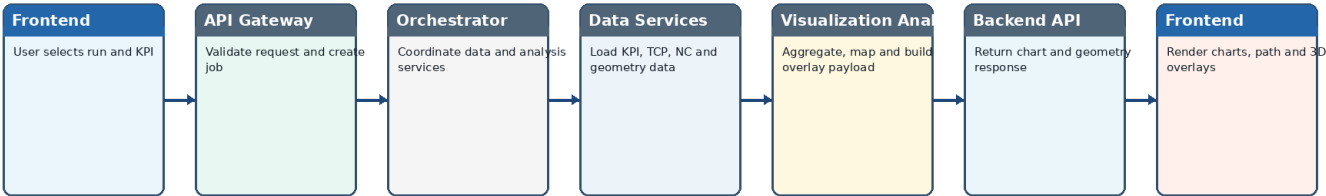
5.1 Level 1 Building Blocks

Building block	Responsibility
Data Services	Retrieve historical and live data, resolve run context, align timestamps and expose normalized records.

Visualization Analytics	Compute dashboard KPIs, TCP paths, planned-versus-actual deviations, geometry associations and visualization payloads.
Advanced Analytics	Perform time-series anomaly analysis, expected-behavior estimation, residual analysis and investigation-context generation.
Orchestrator / Process Master	Sequence workflows, manage service dependencies, monitor jobs, handle errors and aggregate results.
HMI Backend	Expose APIs, validate requests, shape responses, manage live subscriptions and enforce access policies.
HMI Frontend	Provide dashboards, trends, 3D visualization, overlays, investigation views and historical replay.
Cross-Cutting Services	Provide configuration, logging, monitoring, security, schema versioning and recovery capabilities.

6. Runtime View

TYPICAL RUNTIME FLOW - KPI AND GEOMETRY VISUALIZATION



Advanced analytics are optional participants in this flow. Basic KPI, TCP and geometry visualization can be released before anomaly detection.

Figure 3 - Typical KPI and geometry visualization runtime flow

6.1 Historical Run Visualization

When a user selects a historical run, the frontend requests the run summary and available signals from the backend. The API gateway validates the request and delegates it to the orchestrator. The orchestrator retrieves historical KPI, TCP, event, NC and geometry information through the data services. Visualization analytics calculate summaries, prepare the trajectory, perform spatial association and build the chart and overlay payloads. The backend returns a normalized response, after which the frontend updates its state and renders the selected views.

6.2 Live Process Visualization

RXU Connectivity publishes live process values and state changes. The live-data adapter subscribes to the configured topics, associates each message with the active run and forwards normalized updates to the backend streaming gateway. Authorized frontend clients receive the updates through WebSocket subscriptions. The frontend updates KPI cards, trend windows, process state and TCP indicators without reloading the entire page.

6.3 Process Anomaly Investigation

The time-series service detects or evaluates an abnormal condition and returns an anomaly event containing the signal, time interval, score, severity and process context. The orchestrator requests related signal windows, event history and geometry mapping. The investigation service then generates associated variables and contextual evidence. The backend returns a consolidated payload that allows the frontend to show the anomaly on the timeline, highlight its spatial location and display the relevant before-and-after trends.

6.4 Expected-versus-Actual Analysis

For a selected run or interval, the system-identification service evaluates the empirical behavioral model using the relevant process inputs and state. It produces expected response values or an expected envelope. The actual measurements are aligned to these results and residuals are calculated. The frontend receives actual, expected, envelope and deviation series and presents them as a comparison rather than as a claim of defect prediction.

7. Deployment View

The target deployment is an industrial PC located at the edge of the production network. The IPC hosts the frontend web application, backend interface, orchestration process and WP5.3 analysis services. RXU Connectivity may run on the same IPC or on an adjacent edge node depending on the established plant architecture. Historical data may reside locally or in a WP5.1-managed repository accessible over the OT network.

Deployment unit	Typical content	Communication
Frontend container or static web server	Compiled web application and assets	HTTPS/WSS to backend
Backend/API container	REST API, WebSocket gateway, authentication and response shaping	Internal service calls and repository access
Orchestrator process	Workflow and behavior-tree logic	RXU services/topics or internal RPC
Data-service container	Repository adapters, live subscribers and context services	RXU.Data and RXU Connectivity
Analytics containers	Visualization analytics, time-series, system identification and investigation services	Internal APIs or RXU services
Local infrastructure	Cache, logs, configuration and optional local database	Local IPC communication

Containerization is recommended to isolate dependencies and simplify versioned deployment. A reverse proxy shall terminate HTTPS, serve the frontend and route API and WebSocket traffic. Health checks shall be

exposed for all critical services. The IPC shall use a controlled dual-network or segmented-network design when both OT and IT connectivity are present.

8. Cross-Cutting Concepts

8.1 Configuration Management

Configuration includes topic names, repository endpoints, KPI definitions, units, geometry references, feature flags, access roles and analysis parameters. Configuration shall be externalized from application binaries and version controlled.

8.2 Logging and Traceability

Every frontend request that initiates a multi-service workflow shall receive a job identifier. Logs from the API gateway, orchestrator and analysis services shall include this identifier so that a complete execution path can be reconstructed.

8.3 Health Monitoring

Health checks shall cover API availability, RXU.Data connectivity, RXU Connectivity subscriptions, orchestrator status, analysis-service readiness, cache status and storage availability.

8.4 Security

The platform shall use authenticated HTTPS and secure WebSocket communication where required. Role-based access shall separate viewer, engineering and administrative capabilities. Critical user actions and configuration changes shall be auditable.

8.5 Schema and API Versioning

Data schemas, RXU messages and frontend APIs shall be explicitly versioned. Backward-compatible changes shall be preferred, while incompatible changes shall be introduced through a new versioned endpoint or topic.

8.6 Error Handling

Services shall return structured status, error code, human-readable message, source component and job identifier. The orchestrator shall distinguish retryable connectivity failures from invalid input and deterministic analysis failures.

9. Architectural Decisions

Decision	Rationale	Consequence
Frontend communicates only through backend APIs	Prevents direct coupling to databases and RXU internals.	All visualization data must be exposed through controlled contracts.
REST for queries and WebSocket for live updates	Matches historical request-response and live publish-subscribe patterns.	Backend must manage subscriptions and connection recovery.
Analytics services remain independent	Allows statistical, SI and ML implementations to evolve separately.	Adapters and stable result schemas are required.

Orchestrator coordinates multi-step workflows	Centralizes sequencing, progress and failure handling.	Orchestrator becomes a critical monitored component.
Geometry is used for spatial visualization, not physical simulation	Aligns with scope and avoids misleading digital-twin claims.	No dynamic robot or environment simulation is required.
Root-cause capability is investigation support	Reflects limited labeled causal evidence.	UI wording must avoid claiming confirmed automatic diagnosis.

10. Quality Requirements

Quality attribute	Requirement intent	Verification approach
Performance	Dashboard and historical-query response shall be suitable for interactive engineering use; large geometry and time-series data shall be paged, aggregated or streamed.	Load tests and representative dataset benchmarks.
Reliability	Temporary loss of live connectivity shall not crash the HMI; the system shall reconnect and expose status.	Fault-injection and reconnect tests.
Maintainability	Services shall be modular, documented and independently testable.	Code review, interface tests and dependency checks.
Extensibility	New KPIs, overlays and analytics outputs shall be added through configuration and versioned schemas.	Change-impact demonstration.
Usability	Users shall navigate from summary to detailed evidence with consistent filters and context.	User acceptance testing.
Traceability	Every displayed analysis result shall be traceable to run, timestamp, source data and analysis version.	Audit and data-lineage tests.
Security	Only authorized users and systems shall access data and configuration.	Authentication, authorization and penetration checks as applicable.

11. Risks and Technical Debt

Risk	Impact	Mitigation
Late or incomplete WP5.1 data	Delays analytics validation and actual-data integration.	Use sample data, synthetic interface tests and explicit data contracts.

Timestamp misalignment	Incorrect KPI-to-TCP or anomaly-to-geometry mapping.	Define clock source, tolerances and synchronization quality indicators.
Large CAD/mesh files	Slow startup and poor frontend rendering.	Preprocess meshes, use level of detail and cache geometry.
Unclear process states	Incorrect comparisons and false anomaly interpretation.	Validate state semantics with process experts and store state provenance.
Root-cause overclaim	Loss of stakeholder trust.	Present evidence and possible contributors; retain human validation.
Architecture drift between RXU and HMI teams	Integration rework.	Maintain interface control documents and automated contract tests.

12. Detailed Backend Architecture

The backend is the controlled integration boundary between the frontend, RXU platform, data repositories and analysis services. It contains no rendering logic and shall not expose internal database structures directly to the user interface.

12.1 Backend Interface and API Gateway

Receives HTTPS and WebSocket traffic from the frontend. It validates authentication, request syntax, run identifiers, filter parameters and payload size. Historical queries are routed to data services, while analysis requests are converted into orchestrated jobs. Responses are normalized so the frontend does not depend on the internal representation of RXU messages or database records.

12.2 Run Management Service

Maintains the identity and metadata of production runs, including part, program, layer, course, start and end time, available signals and geometry reference. It provides the anchor used to correlate historical records, live messages and analysis outputs.

12.3 Historical Data Service

Retrieves validated process data from RXU.Data or the WP5.1 repository through a repository abstraction. It supports run-, time-, signal-, layer- and course-based queries and returns normalized records to other services.

12.4 Live Data Service

Subscribes to RXU Connectivity topics, manages reconnect behavior and converts incoming messages into the internal canonical schema. It tracks the active run context and forwards live updates to subscribed clients through the backend streaming gateway.

12.5 Data Context and Synchronization Service

Associates KPI, TCP, events and NC records through timestamp, run and process-state context. It performs WP5.3-specific alignment and enrichment but does not duplicate the primary cleaning and validation already performed by WP5.1.

12.6 Visualization Analytics Service

Calculates KPI summaries, resampling for charts, TCP trajectories, run summaries and planned-versus-actual differences. It also produces compact payloads for charts, tables, timelines and geometry overlays.

12.7 Geometry Mapping Service

Loads or references the approved part representation, resolves coordinate-system metadata and associates TCP points, KPI values and analysis events with spatial locations. It exposes geometry, overlay and point-inspection data through backend APIs.

12.8 Analytics Integration Service

Adapts time-series anomaly, system-identification and root-cause investigation outputs to the common HMI schema. It allows algorithms to evolve without requiring frontend changes.

12.9 Orchestrator / Process Master

Coordinates workflows spanning multiple services. It manages prerequisites, job state, progress, timeout, retry and partial-failure behavior. A behavior tree or equivalent workflow model is suitable where the RXU architecture already uses this pattern.

12.10 Cache and Storage Adapter

Caches frequently requested run summaries, chart aggregates and geometry overlays. Persistent analysis results may be stored with their model and algorithm version to support traceability.

12.11 Configuration, Logging and Health Services

Provide centralized runtime configuration, structured logging, audit records, service-health reporting, backup information and deployment diagnostics.

13. Detailed Frontend Architecture

The frontend is a browser-based engineering HMI designed for progressive delivery. Version 1 establishes the design system, navigation, screen templates and frontend-to-backend connectivity. Later releases add real KPI and TCP data, geometry overlays, anomaly investigation and behavioral-model views.

13.1 Application Shell and Navigation

Provides the common header, navigation menu, route handling, user context, backend status and consistent screen layout. The shell shall preserve selected run and filter context when the user moves between views.

13.2 Dashboard

Shows run identity, process state, data availability, KPI summary cards, active events and links to detailed views. In early releases, the dashboard may use mock payloads while preserving the final API contract.

13.3 KPI Trends and Comparison

Displays one or more KPI series with units, time range, cursor inspection, zoom and process-state context. It supports synchronized cursors across multiple charts so related changes can be investigated.

13.4 Process Timeline and Event View

Presents process states, run phases, machine events and analysis events on a common time axis. Selecting an event updates related charts and geometry context.

13.5 Three-Dimensional Geometry Viewer

Displays the approved part mesh with rotate, pan, zoom, reset, layer visibility and selection. It shall render actual TCP paths, programmed paths, KPI overlays, anomaly markers and selected regions without implying physical simulation.

13.6 Overlay Management

Allows the user to select the displayed KPI, color scale, path, anomaly layer and time or run interval. Tooltips shall show the mapped timestamp, TCP pose, signal value and process context.

13.7 Anomaly Investigation View

Combines anomaly metadata, before-and-after trends, process-state events, geometry location and associated variables. It is enabled when time-series analysis results are available.

13.8 Expected-versus-Actual View

Displays expected response, actual response, expected envelope and residual or deviation. It is enabled when a validated behavioral model produces the required output.

13.9 Historical Run Explorer

Supports run selection, replay, comparison and retrieval of historical overlays. The user may move through time while charts, events and TCP position remain synchronized.

13.10 Frontend Communication Layer

Encapsulates REST clients, WebSocket subscriptions, authentication tokens, retry logic, error handling and response conversion. UI components shall not call endpoints directly outside this layer.

14. Analytics and Process Twin Services

14.1 Process Data Analysis for Visualization

This service can be delivered before time-series anomaly analysis and system identification. It consumes the dataset prepared by the preceding data-preparation deliverable and generates engineering and visualization products. Its outputs include KPI aggregates, TCP trajectories, planned-versus-actual path and speed comparisons, process timelines, geometry mappings, heatmap values and HMI-ready payloads.

14.2 Time-Series Process Anomaly Analysis

The time-series service evaluates temporal behavior to identify threshold violations, spikes, drift, oscillation, change points or statistically unusual windows. Its primary subject is process anomaly rather than product defect. The service shall generate traceable anomaly events with signal, time interval, severity, score, method, process state and spatial association where available.

14.3 System Identification and Behavioral Model

The system-identification service learns or applies empirical input-output relationships using process signals and state. It estimates expected process behavior under given conditions and supports expected-versus-actual comparison. It does not require a 3D environment model and shall not be described as a physics-exact digital twin. Candidate methods may include ARX, state-space, regression and other data-driven temporal models selected through validation.

14.4 Root-Cause Investigation Support

The investigation service combines anomaly events, related KPI windows, process-state transitions, planned-versus-actual deviations, behavioral residuals and spatial context. It ranks or presents possible contributing variables and associated evidence. The service must preserve the distinction between correlation, temporal association and confirmed causation.

15. Data and Interface Architecture

15.1 Canonical Data Entities

Entity	Key attributes
Run	runId, partId, programId, startTime, endTime, machineId, geometryId, dataStatus
KPI sample	runId, timestamp, signalId, value, unit, processState, qualityFlag, source
TCP pose	runId, timestamp, position, orientation, coordinateFrame, processState
NC point/path	programId, sequence, plannedPose, plannedSpeed, layer, course
Event	eventId, runId, timestamp, type, severity, state, description
Anomaly	anomalyId, runId, signal, start/end, score, severity, method, context
Behavior result	runId, modelVersion, expectedValue/envelope, actualValue, residual, timestamp
Geometry overlay	geometryId, overlayType, points/regions, values, scale, time range

15.2 API Categories

API category	Representative capability
Run APIs	List runs, retrieve metadata and available signals.
KPI APIs	Retrieve summary cards, trend series and chart aggregates.
Trajectory APIs	Retrieve actual TCP, programmed path and comparison results.
Geometry APIs	Retrieve geometry metadata, overlays and point-inspection information.
Timeline APIs	Retrieve process states and events.
Anomaly APIs	Retrieve anomaly lists, details and investigation context.

Behavior APIs	Retrieve expected-versus-actual data, envelopes and residuals.
Job APIs	Start analysis, query progress and retrieve result status.
Health APIs	Expose backend and dependency status.

15.3 Communication Rules

REST responses shall include schema version, request or job identifier, status, data and structured error information. WebSocket messages shall identify subscription, message type, run, timestamp and sequence where ordering matters. Internal RXU topics and services shall use stable names, explicitly defined units and documented quality flags.

16. Testing and Validation Strategy

Test level	Scope
Unit testing	Parsing, mapping, KPI calculations, API validation, state reducers and UI components.
Contract testing	WP5.1 data contracts, RXU topics/services and frontend-backend APIs.
Integration testing	Historical retrieval, live subscription, orchestration, analytics adapters and geometry mapping.
System testing	End-to-end runs using representative data, including API, charts, timeline and 3D overlays.
Performance testing	Large time-series queries, multi-run history, geometry loading and live update rates.
Usability testing	Navigation, filtering, context preservation and anomaly-investigation workflows.
Acceptance testing	Stakeholder-approved scenarios for each HMI release and analytics version.

Validation shall distinguish software correctness from analytical validity. A correctly displayed anomaly is not necessarily a meaningful process anomaly unless the underlying method and threshold have been validated. Similarly, a system-identification model shall be accepted only after residual, stability and generalization assessment using representative data.

17. Release and Evolution Plan

Release	Content	Dependencies
HMI Version 1 - Framework and navigation prototype	Application shell, landing page, buttons, navigation, placeholder screens, sample KPI cards, mock data, basic 3D placeholder and frontend-backend connectivity.	Architecture and design freeze; mock API contracts.
HMI Version 2 - Process data visualization	Actual KPI trends, run selection, process timeline, TCP trajectory, NC path, basic part geometry and KPI overlays.	WP5.1 data access, visualization analytics and geometry mapping.

HMI Version 3 - Process intelligence integration	Process anomaly views, expected-versus-actual views, investigation context, historical replay and mature overlays.	Validated time-series, system-identification and investigation outputs.
--	--	---

The versioned strategy avoids blocking HMI development on analytics maturity. Each release shall preserve API contracts where possible and introduce advanced functionality as an additional layer rather than rewriting the basic application structure.

18. Glossary

Term	Definition
Process Twin	Data-centric representation of process execution, spatial context, abnormal behavior and expected behavior; not a physics-based environment simulation.
HMI	Human-machine interface used here for process visualization and engineering investigation.
R XU	Application and connectivity framework used for internal services, orchestration and data exchange.
WP5.1	Work package responsible for acquisition, cleaning, filtering, validation and data availability.
KPI/KPP	Process or performance indicator supplied or calculated for analysis and visualization.
TCP	Tool center point pose recorded from the machine or robot.
Process anomaly	Abnormal signal or process behavior such as spike, drift, oscillation or deviation; not automatically a product defect.
System identification	Data-driven estimation of process input-output behavior.
Residual	Difference between actual and expected model output.
Root-cause investigation support	Contextual evidence and possible contributing variables provided to engineers; not necessarily confirmed diagnosis.

Appendix A. Reference Architecture Diagrams

The following project diagrams were used as references for the formal architecture described in this specification.

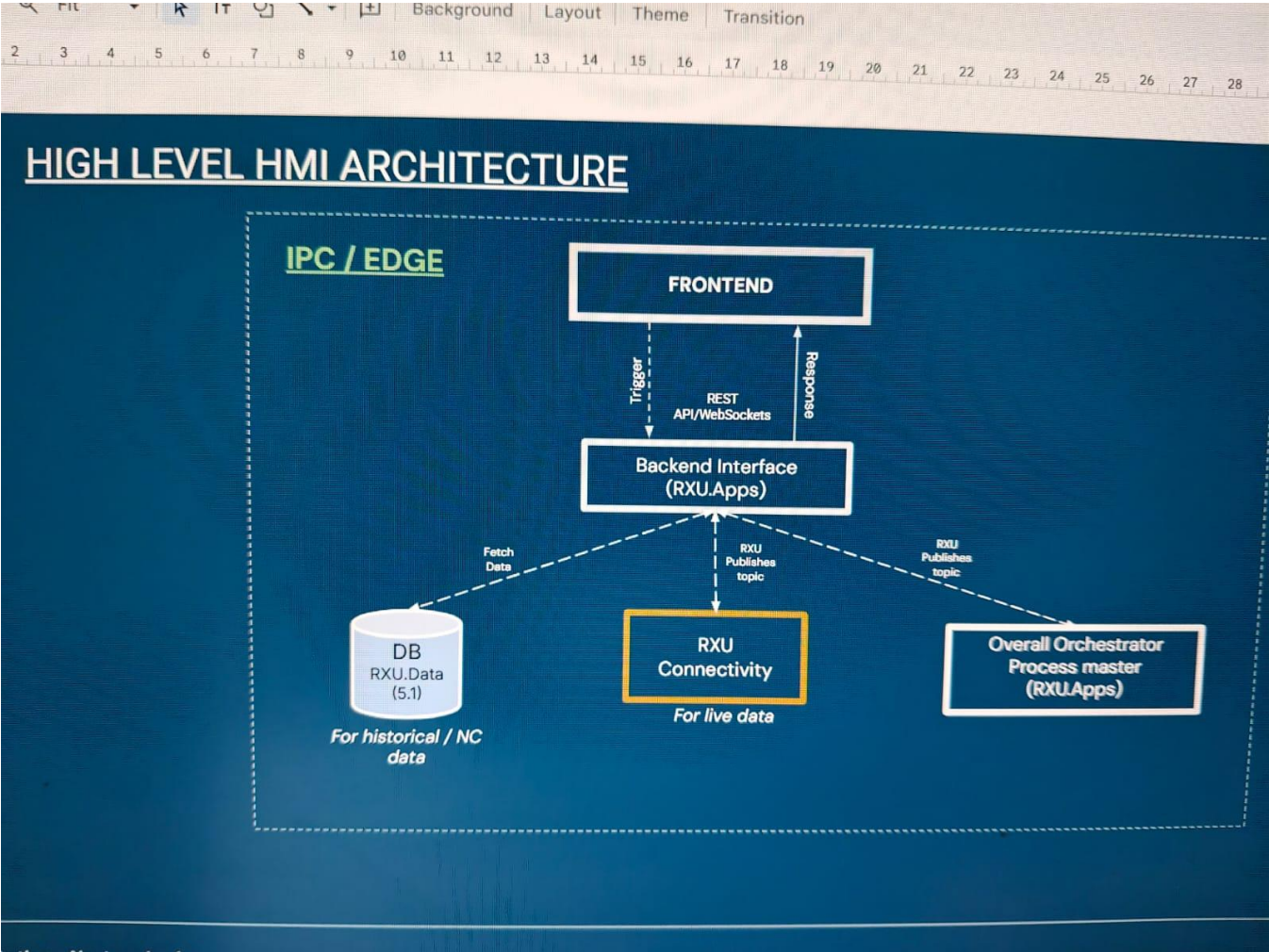


Figure A-1 - Project high-level HMI architecture reference

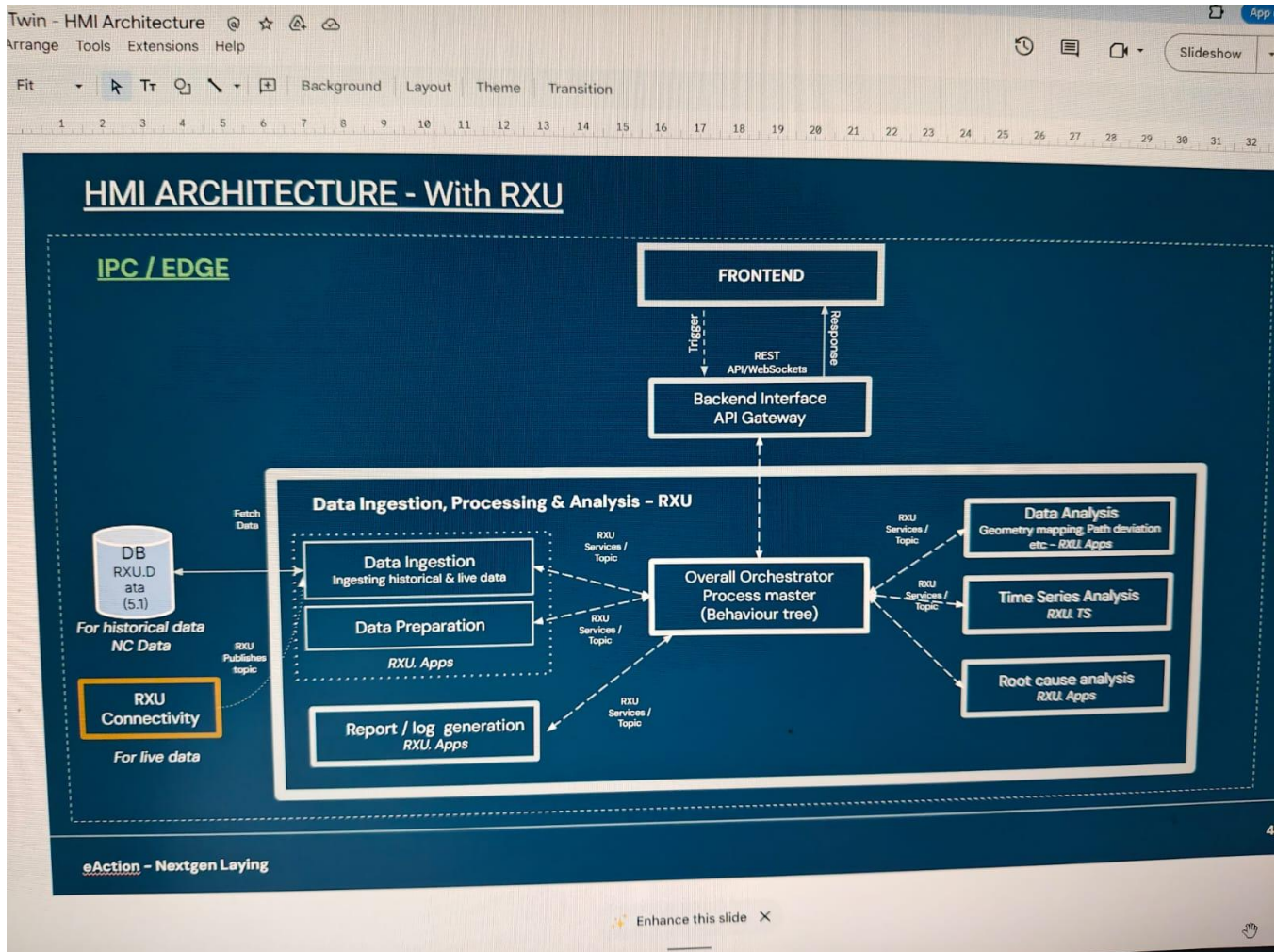


Figure A-2 - Project detailed HMI architecture reference

Appendix B. Example API and Message Contracts

B.1 KPI Trend Response

A KPI trend response shall identify the schema version, run, signal, unit, time range and ordered samples. Optional process-state and quality flags should be included when they are required by the HMI.

```
{
  "schemaVersion": "1.0",
  "runId": "RUN-2026-001",
  "signal": "tow_tension",
  "unit": "N",
  "samples": [
    {"timestamp": "2026-07-15T10:15:32.125Z", "value": 21.4, "processState": "laying"}
  ]
}
```

B.2 Geometry Overlay Response

```
{
  "schemaVersion": "1.0",
```

```
"runId": "RUN-2026-001",  
"geometryId": "PART-A-MESH-V3",  
"overlayType": "kpi_path",  
"signal": "tow_tension",  
"points": [  
  {"x": 1.24, "y": 0.42, "z": 2.11, "value": 21.4, "timestamp": "2026-07-15T10:15:32.125Z"}  
]  
}
```